

Automated Character Identification in Multi-Cast Cinematic Universes

Course:	Programming for AI
Group:	10 (Project 2)
Members:	M. Fahad Pasha (BSCS24147) - M. Ali (BSCS24073)
Domain:	Computer Vision – Deep Learning

1 Introduction

1.1 Motivation: Challenges and Applications

Streaming platforms such as Netflix, Disney+, and Amazon Prime Video collectively serve over 700 million subscribers worldwide, producing content libraries spanning thousands of hours of footage per year. Annotating cast members across this volume of material manually is prohibitively expensive and does not scale. Automated character identification through deep learning offers a tractable solution but the cinematic domain introduces challenges that are absent from controlled biometric benchmarks:

Key Challenges	Real-World Applications
Intra-class appearance variation: A character's appearance can shift dramatically across seasons e.g., Eleven's buzz cut in S1 versus long hair in S3 creating high within-class feature variance.	Automated content indexing: Streaming platforms (Netflix, Prime Video) can auto-tag frames with cast metadata, reducing editorial annotation costs and enabling rapid content search.
Inter-class visual similarity: Characters sharing similar demographic attributes (skin tone, hair color, build) cause high false-positive rates in naive classifiers.	Personalized thumbnail generation: Netflix A/B testing research has shown that thumbnails featuring a viewer's preferred character increase click-through rate by up to 20–30%, requiring accurate cast detection.
Unconstrained pose and occlusion: Cinematic shots frequently include partial face occlusion, extreme pose angles, and motion blur that degrade recognition performance.	Interactive second-screen features: Real-time character identification enables 'X-Ray'-style overlays (as used on Amazon Prime Video) that display actor and character information during playback.
Dramatic and non-uniform illumination: Night scenes, coloured practical lights, and high-contrast thriller lighting in Stranger Things shift perceived skin tone and facial structure, degrading feature stability.	Film archive and rights management: Studios and broadcasters can audit screen-time per character automatically, supporting contract compliance, royalty calculations, and content rights workflows.

1.2 Problem Statement

This project develops a deep learning pipeline for closed-set character identification in raw, uncured cinematic imagery automatically classifying five recurring characters from the Stranger Things universe using a transfer-learned ResNet-50 model trained exclusively on self-collected data, without reliance on any pre-labelled public dataset.

2 Background / Literature Review

The following four works represent the direct lineage of methodology adopted in this proposal, spanning foundational face recognition architectures to state-of-the-art margin-based loss functions and cinematic-specific face clustering.

[1] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. CVPR 2015.

FaceNet introduced a CNN-based approach that maps facial images directly into a 128-dimensional Euclidean embedding space, such that L2 distances correspond to face similarity. Its triplet loss formulation ensures that an anchor image is closer to all positive samples (same identity) than to any negative sample by a margin. *Relevance to this project:* the triplet loss concept directly informs the contrastive training strategy considered for handling high intra-class variation across Stranger Things seasons.

[2] Kim, M., Jain, A. K., & Liu, X. (2022). AdaFace: Quality Adaptive Margin for Face Recognition. CVPR 2022 (Oral).

AdaFace extended the margin-based softmax lineage established by ArcFace (Deng et al., 2019) by introducing an adaptive margin function that adjusts based on estimated image quality, approximated via feature norm magnitude. For low-quality images, the loss de-emphasises hard samples (avoiding training on unidentifiable faces); for high-quality images, it emphasises them. AdaFace surpassed ArcFace on all four major benchmarks: IJB-B, IJB-C, IJB-S, and TinyFace. *Relevance:* Cinematic footage exhibits highly variable frame quality, motion blur, extreme lighting, and dark scenes are endemic to Stranger Things. AdaFace's quality-aware loss is therefore a stronger candidate than fixed-margin ArcFace for this dataset and is adopted as the experimental loss variant in Section 4.

[3] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.

ResNet demonstrated that very deep networks (50–152 layers) could be trained stably by introducing skip connections that allow gradients to flow through identity mappings. ResNet-50 pretrained on ImageNet provides rich, hierarchical feature representations for facial texture, edge, and structural patterns. *Relevance:* ResNet-50 is selected as the backbone architecture for this project. Transfer learning from ImageNet weights substantially mitigates the modest dataset size (~1,250–1,500 images) by leveraging pre-learned low- and mid-level visual features.

[4] Brown, A., Xie, W., Kalogeiton, V., & Zisserman, A. (2021). Face Grouping in the Wild. BMVC 2021.

This work addressed the specific problem of face clustering and identity grouping in unconstrained cinematic footage, proposing a GNN-based approach to link face tracklets across scenes. It explicitly handled the challenges of occlusion, lighting change, and age variation across film productions. *Relevance:* This is the most domain-specific citation in the review, directly addressing character identification in the cinematic setting. It establishes that appearance-based classifiers alone are insufficient at scale, motivating the use of strong margin losses and augmentation strategies in this project.

3 Dataset Details

In compliance with the course requirement, no pre-processed or publicly available labelled datasets will be used. All data will be self-collected from raw, unlabeled source material.

3.1 Classes and Volume

Character	Target Images	Seasons Source	Notes
Eleven	250 – 300	S1 – S5	High variance: buzz cut → long hair
Dustin	250 – 300	S1 – S5	Distinct cap (Strong visual anchor)
Steve	250 – 300	S1 – S5	Similar hair/build to Jonathan (Key confusion pair)
Hopper	250 – 300	S1 – S5	Sheriff uniform aids detection across seasons
Lucas	250 – 300	S1 – S5	Frequently co-appears with Dustin (Occlusion Risk)
Total	1,250 – 1,500	-	250 per class minimum

3.2 Collection Method

- **Frame extraction from HD video:** Automated frame sampling at 1 frame/second from publicly available HD trailers, featurettes, and promotional clips using OpenCV's VideoCapture API. Frames with motion blur (Laplacian variance < 80) will be discarded.
- **Web scraping:** Targeted scraping of press release images and promotional stills via BeautifulSoup and Selenium. Duplicate images removed via perceptual hashing (pHash, Hamming distance threshold = 10).
- **Manual review pass:** All collected images reviewed manually to ensure correct character labelling before training.

3.3 Train / Validation / Test Split

The dataset will be divided using a stratified 70% / 15% / 15% split (train / validation / test) to maintain class balance across all partitions. The test set will be held out entirely until final evaluation.

3.4 Data Augmentation Strategy

To compensate for dataset size and improve generalization, the following augmentation pipeline will be applied to training images using the Albumentations library:

Augmentation	Parameters	Rationale
Horizontal flip	p = 0.5	Pose symmetry
Random rotation	±15°	Camera tilt variation
Color jitter	brightness ±0.3, hue ±0.1	Lighting + scene color
Gaussian noise	var = 0.005 – 0.02	Simulate compression artefacts
Random crop + resize	80–100% of original	Scale invariance
CutOut	2 patches, 20×20 px	Occlusion robustness

4 Proposed Methodology

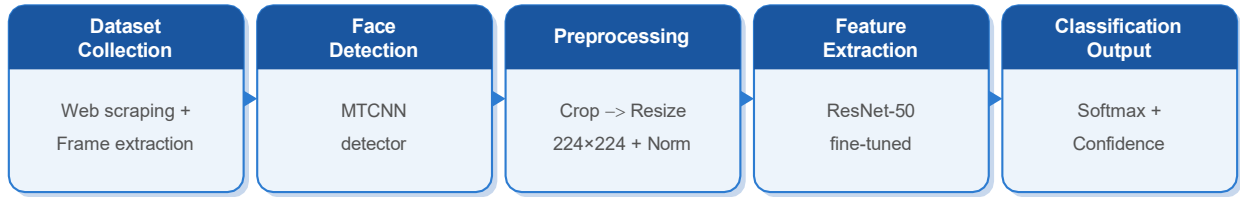


Figure 1: Complete system pipeline

4.1 Stage 1 - Dataset Collection

Raw video frames are extracted at 1 fps using OpenCV and combined with web-scraped promotional images. Blur detection (Laplacian variance threshold) and perceptual deduplication (pHash) are applied before manual labelling review.

4.2 Stage 2 - Face Detection (MTCNN)

Each collected image is passed through the Multi-Task Cascaded Convolutional Network (MTCNN) face detector (Zhang et al., 2016), which produces bounding boxes and five facial landmark coordinates (eyes, nose, mouth corners). MTCNN is selected over Haar cascades due to its superior performance on occluded and non-frontal faces. Only images yielding exactly one detected face are retained; multi-face frames are discarded to avoid ambiguous labels.

4.3 Stage 3 - Preprocessing

- **Landmark-aligned crop:** Five-point affine alignment normalizes head pose before cropping, reducing pose-induced feature shift.
- **Resize:** All crops resized to 224×224 pixels to match ResNet-50 input dimensions.
- **Normalization:** Pixel values normalized to ImageNet mean ($\mu = [0.485, 0.456, 0.406]$) and standard deviation ($\sigma = [0.229, 0.224, 0.225]$) for compatibility with pretrained weights.

4.4 Stage 4 - Model Architecture (ResNet-50 + Fine-tuning)

A ResNet-50 backbone pretrained on ImageNet will be loaded via PyTorch's torchvision model zoo. Transfer learning will proceed in two phases:

- **Phase 1 - Feature extractor (5 epochs):** All convolutional layers frozen; only the final fully-connected (FC) classification head (replaced with Linear(2048→5) + Softmax) is trained. Learning rate: 1×10^{-3} , Adam optimizer.
- **Phase 2 - Full fine-tuning (20 epochs):** Layers from ResNet block layer3 onward unfrozen. Learning rate reduced to 1×10^{-4} with cosine annealing scheduler. Dropout ($p=0.4$) added before FC layer to reduce overfitting.

Loss function: Cross-Entropy Loss as the baseline, with AdaFace quality-adaptive margin loss as an experimental variant, motivated by [2]. Batch size: 32. Early stopping: patience = 5 epochs on validation loss.

4.5 Stage 5 - Output and Post-Processing

The final softmax layer produces a probability distribution across five character classes. The predicted class is the argmax of this distribution. A confidence threshold ($\tau = 0.70$) will be applied: predictions below τ are flagged as 'uncertain' and excluded from evaluation metrics. This threshold is determined empirically on the validation set.

4.6 Evaluation Metrics

Model performance will be assessed on the held-out test set using the following metrics:

Metric	Formula / Tool	Purpose
Top-1 Accuracy	Correct / Total	Overall correctness
Per-class F1-Score	$2 \times P \times R / (P + R)$, macro average	Class-level balance
Confusion Matrix	<code>sklearn.metrics.ConfusionMatrixDisplay</code>	Error pattern analysis
ROC-AUC (OvR)	<code>sklearn roc_auc_score</code> , multi-class	Probability calibration

Everything will be implemented in Python 3.10 using PyTorch 2.x, torchvision, OpenCV, Albumentations, and scikit-learn. Code and dataset collection scripts will be version-controlled via Git.